

Teoria do dashboard

wd-magnetizada

O que cada aba calcula: equações, código
e as decisões que custaram caro

23 de Julho de 2026

Anã branca com campo magnético interior misto (poloidal + toroidal)
Documento de referência técnica — não é introdução a física estelar nem a MHD

Conteúdo

Tabela de símbolos e unidades	3
1 Núcleo teórico comum	4
1.1 Equação de estado	4
1.2 Gravidade própria	5
1.3 Campo magnético e a função de fluxo	5
1.4 Grad-Shafranov	6
1.5 Bernoulli e a parametrização	7
1.6 O loop SCF	8
1.7 Virial e diagnósticos de energia	9
1.8 As duas razões B_t/B_p	10
1.9 O campo toroidal e o twisted torus	10
1.10 Escalas de tempo e unidades	11
2 ABA 1 — Equilíbrio	13
3 ABA 2 — Varredura	14
4 ABA 3 — Exportação	15
5 ABA 4 — Registro	16
6 Limitações conhecidas	16
7 Questões em aberto	17
8 Referências	18

Este documento explica o que o dashboard calcula: as equações por trás de cada número e cada figura, e o código que as implementa. Não é uma introdução a física estelar nem ao MHD — pressupõe que o leitor já conhece isso. O que este documento faz é a ponte entre a teoria (seção 1) e o código real (`scf/`, `dashboard/`).

Ver `plano_wd_magnetizada.md` para o plano de projeto completo (decisões D1–D6, arquitetura, fases). Este documento é sobre a *física implementada*, não sobre o plano de trabalho.

Tabela de símbolos e unidades

Símbolo	Significado	Unidade (CGS)	Nome no código
r, θ	coordenadas esféricas	cm, rad	<code>r, theta</code>
ϖ	raio cilíndrico, $\varpi = r \sin \theta$	cm	<code>omega</code> (ver nota)
z	altura, $z = r \cos \theta$	cm	<code>z</code> (só em <code>plots.py</code>)
ρ	densidade de massa	g cm^{-3}	<code>rho</code>
x	momento de Fermi normalizado, $x = p_F/(m_e c)$	adimensional	<code>x</code>
P	pressão	dyn cm^{-2}	<code>pressure(x)</code>
H	entalpia específica	erg g^{-1}	<code>H</code>
Φ	potencial gravitacional	erg g^{-1}	<code>Phi</code>
A_φ	componente φ do potencial vetor	G cm	<code>A_phi</code>
u	função de fluxo, $u = \varpi A_\varphi$	G cm^3	<code>u</code>
B_r, B_θ, B_φ	componentes do campo magnético	G (gauss)	<code>Br, Bth/Btheta, Bphi</code>
$f(u)$	função de corrente poloidal, $f(u) = k_0$	$\text{g}^{-1/2} \text{cm}^{1/2} \text{s}^{-1}$	<code>k0</code>
$M(u)$	potencial de Bernoulli do poloidal	erg g^{-1}	<code>M_u</code> (local em <code>scf.py</code>)
$\beta(u)$	função de corrente toroidal, $\beta = \varpi B_\varphi$	G cm	não calculada à parte
ζ	amplitude do toroidal imposto	—	<code>zeta</code>
m	expoente do toroidal imposto	inteiro ≥ 1	<code>m_tor</code>
u_c	valor de u na última linha fechada	G cm^3	<code>u_c</code>
C	constante de Bernoulli	erg g^{-1}	<code>C</code>
ρ_c	densidade central (parâmetro de entrada)	g cm^{-3}	<code>rho_c</code>
μ_e	peso molecular médio por elétron, $Y_e = 1/\mu_e$	adimensional	<code>mu_e</code>
M	massa total da estrela	g (exibido em $M_\odot$)	<code>M</code> (colide com $M(u)$ — ver nota)
R_{eq}, R_{pol}	raios equatorial e polar (onde $H = 0$)	cm (exibido em km)	<code>R_eq, R_pol</code>
W	energia gravitacional	erg	<code>W</code>
Π	energia interna, $\int P dV$	erg	<code>Pi</code>
$E_{pol}, E_{tor}, \mathcal{M}$	energias magnéticas	erg	<code>E_pol, E_tor, E_mag</code>
VE	erro virial	adimensional	<code>VE</code>
G	constante gravitacional	$6,674 \times 10^{-8}$ cgs	<code>G_CONST</code>
A, B	constantes da EOS	ver §1.1	<code>A_CONST, B_of_mu_e()</code>
l	grau da expansão de Legendre	inteiro $\geq 0/\geq 1$	<code>l</code>
t_{din}, v_A, t_{Alf}	escalas derivadas	s, cm/s, s	<code>t_din, v_A, t_alf</code>

Notas de notação:

- O código usa a variável **omega** para ϖ (raio cilíndrico), **não** para velocidade angular — não há rotação neste projeto (D3, estrela estática). É uma escolha de nome específica deste código; se ler **omega** em `scf.py/gradshafranov.py`, é sempre $\varpi = r \sin \theta$.
- **M** é usado na espinha teórica para dois objetos diferentes: a massa estelar total e o potencial $M(u)$ do Bernoulli (§1.5). O código resolve a colisão nomeando a massa **M** (retornada por `scf.total_mass()`) e o potencial **M_u** (variável local dentro de `hachisu_scf()`, nunca exposta fora da função). Neste documento, sempre que houver risco de ambiguidade, o texto diz “massa” ou “potencial de Bernoulli” por extenso.
- **Bth** aparece como **Btheta** em `diagnostics.py` (parâmetro da função) e **Bth** nas páginas do dashboard (variável local) — mesmo objeto, dois nomes.

1 Núcleo teórico comum

1.1 Equação de estado

Gás de elétrons completamente degenerado a $T = 0$:

$$P = A \left[x(2x^2 - 3)(x^2 + 1)^{1/2} + 3 \sinh^{-1} x \right] \quad (1)$$

$$\rho = Bx^3 \quad (2)$$

$$x \equiv \frac{p_F}{m_e c} \quad (3)$$

$$A = 6,01 \times 10^{22} \text{ dyn cm}^{-2} \quad (4)$$

$$B = \frac{9,82 \times 10^5}{Y_e} \text{ g cm}^{-3} \quad (Y_e = 0,5 \Rightarrow B \approx 1,96 \times 10^6) \quad (5)$$

A é uma constante física fixa (combinação de m_e , c , $\hbar$); B depende da composição só através de Y_e (número de elétrons por núcleon). O dashboard parametriza por $\mu_e = 1/Y_e$ em vez de Y_e diretamente.

→ `scf/eos.py :: pressure()`

(`P(x)`), `density()` ($\rho(x)$), `B_of_mu_e()` ($B(\mu_e)$)

Entalpia específica, analítica — é isso que torna o SCF possível:

$$H = \int \frac{dP}{\rho} = \frac{8A}{B} \left[\sqrt{1 + x^2} - 1 \right] \quad (6)$$

→ `scf/eos.py :: enthalpy()`

Inversa, aplicada a cada iteração do SCF:

$$x = \sqrt{\left(1 + \frac{HB}{8A}\right)^2 - 1} \quad (H \leq 0 \Rightarrow \rho = 0) \quad (7)$$

→ `scf/eos.py :: x_of_enthalpy()`, `density_of_enthalpy()`

O termo -1 normaliza $H = 0$ na superfície (fronteira $\rho = 0$ da estrela).

Índice politrópico efetivo. Perto da superfície $x \ll 1$ dá $\rho \propto H^{3/2}$, isto é $n = 3/2$. No núcleo relativístico $x \gg 1$ dá $\rho \propto H^3$, isto é $n = 3$. A EOS desliza entre os dois, e $n = 3$ é o caso marginal onde a massa fica independente da densidade central — a origem do limite de Chandrasekhar. Esse fato é a causa direta da decisão de parametrização em §1.4/§1.5.

Velocidade do som. Adicionada depois da espinha original (não estava no plano), por ser necessária para a amplitude da perturbação exportada na Aba 3:

$$c_s = \sqrt{\frac{dP}{d\rho}}, \quad \frac{dP}{dx} = \frac{8Ax^4}{\sqrt{1+x^2}} \quad (\text{derivada analítica de } P(x)) \quad (8)$$

→ `scf/eos.py :: sound_speed()`

, validada contra diferença finita em `scf/tests/test_sound_speed.py`

1.2 Gravidade própria

$$\nabla^2 \Phi = 4\pi G \rho \quad (9)$$

Resolvida por expansão em polinômios de Legendre. Com $\rho(r, \theta) = \sum_l \rho_l(r) P_l(\cos \theta)$:

$$\Phi_l(r) = -\frac{4\pi G}{2l+1} \left[r^{-(l+1)} \int_0^r \rho_l(r') r'^{l+2} dr' + r^l \int_r^\infty \rho_l(r') r'^{1-l} dr' \right] \quad (10)$$

Os pesos r'^{l+2} e r'^{1-l} carregam o r'^2 do elemento de volume.

→ `scf/poisson.py :: solve_poisson()`

(implementação exata desta fórmula, D_l/E_l calculados por soma cumulativa trapezoidal); `legendre_matrix()` calcula $P_l(\cos \theta)$.

Validado contra a solução fechada da esfera uniforme e o teorema das cascas de Newton em `scf/tests/test_poisson.py`.

$G = 6,674 \times 10^{-8} \text{ cm}^3 \text{ g}^{-1} \text{ s}^{-2}$ — constante física (valor de referência CODATA arredondado), não um parâmetro de projeto.

→ `scf/poisson.py :: G_CONST`

, repetida em `dashboard/units.py :: G_CONST` (mesmo valor, dois módulos, porque `poisson.py` não pode depender do `dashboard` — R1).

1.3 Campo magnético e a função de fluxo

Com $\varpi = r \sin \theta$, define-se a função de fluxo

$$u = \varpi A_\varphi \quad (11)$$

e o campo axissimétrico separa em poloidal mais toroidal:

$$\mathbf{B} = \frac{1}{\varpi} [\nabla u \times \hat{e}_\varphi + \beta(u) \hat{e}_\varphi] \quad (12)$$

Em componentes esféricas:

$$B_r = \frac{1}{r^2 \sin \theta} \frac{\partial u}{\partial \theta} \quad (13)$$

$$B_\theta = -\frac{1}{r \sin \theta} \frac{\partial u}{\partial r} \quad (14)$$

$$B_\varphi = \frac{\beta(u)}{\varpi} \quad (15)$$

→ scf/diagnostics.py :: poloidal_field()

implementa B_r , B_θ (por diferenças finitas de u , com proteção contra a singularidade de coordenada em $\varpi \rightarrow 0$). B_φ não é calculado a partir de um $\beta(u)$ intermediário; sai diretamente de scf/toroidal.py :: impose_toroidal(), que já implementa a forma funcional escolhida para β (§1.9) substituída na fórmula acima.

A forma geral $\mathbf{B} = \frac{1}{\varpi}[\nabla u \times \hat{e}_\varphi + \beta \hat{e}_\varphi]$ como identidade vetorial não aparece escrita em nenhum lugar do código — o código vai direto às três componentes escalares.

Interpretação geométrica: os contornos de u são as linhas de campo poloidal. É por isso que a Aba 1 os plota (§2).

1.4 Grad-Shafranov

$$\Delta^* u = -4\pi\varpi^2 \rho f(u) - \beta\beta'(u) \quad (16)$$

$$\Delta^* = \frac{\partial^2}{\partial \varpi^2} - \frac{1}{\varpi} \frac{\partial}{\partial \varpi} + \frac{\partial^2}{\partial z^2} \quad (17)$$

$f(u) = dM/du$ é a função de corrente que gera o poloidal; $\beta(u)$ gera o toroidal. A escolha mais simples, e a usada aqui, é $f(u) = k_0$ constante (Lander & Jones 2009). Como o toroidal é imposto depois de o poloidal convergir (§1.9, D6) e não entra na própria equação GS resolvida pelo SCF, o termo $-\beta\beta'(u)$ **não aparece na fonte que o código de fato monta** — a fonte implementada é só $-4\pi\varpi^2 \rho k_0$.

→ scf/gradshafranov.py :: solve_gradshafranov()

— fonte passada por scf/scf.py :: hachisu_scf() como source = -4*np.pi*omega2*rho*k0.

A densidade de corrente que corresponde a essa fonte é

$$J_\varphi = c\rho\varpi f(u) \quad (18)$$

Esta fórmula **não corresponde a nenhuma função no código** — J_φ nunca é calculada como quantidade nomeada. Ela foi usada apenas analiticamente, durante a depuração que achou o bug descrito no quadro abaixo, para montar o lado direito da lei de Ampère integral (testes sem derivada, mais adiante).

Resolução por expansão de Legendre associada. O operador Δ^* separa em funções $P_l^1(\cos\theta)$, **não** em $P_l(\cos\theta)$. A estrutura radial da função de Green é análoga à de Poisson, mas os pesos das integrais **não são os mesmos**:

	peso interno	peso externo
Poisson (para Φ)	r'^{l+2}	r'^{1-l}
Grad-Shafranov (para u)	r'^{l+1}	r'^{-l}

→ scf/gradshafranov.py :: solve_gradshafranov()

Nas linhas onde D_l/E_l são montados com $r^{** (l + 1)}$ e $r^{** (-l)}$.

Por que assim — os expoentes da função de Green

A diferença de uma potência vem do peso em ϖ da fonte da GS comparado ao r'^2 do elemento de volume em Poisson, e depende de a equação ser resolvida para u ou para A_φ .

Este foi um bug real neste projeto: uma versão anterior de `gradshafranov.py` usava r'^{l+2} e r'^{1-l} (copiados por engano da estrutura de `poisson.py`, que tem uma potência de r a mais por causa da substituição $\chi = r\Phi_l$ usada na redução do Laplaciano escalar — Δ^* não precisa dessa substituição). O bug inflava o campo poloidal por um fator $\sim 7000\times$ e a energia magnética por $\sim 5 \times 10^7$.

O bug sobreviveu a toda a validação peça-a-peça — inclusive uma forma fechada derivada “à mão” — porque essa forma fechada foi construída reusando a mesma equação indicial (r^{l+1} , r^{-l} para as soluções homogêneas, que **estavam certas**) e por isso herdava a mesma normalização das integrais internas, que estava errada. Só dois testes que não usam a função de Green nem uma segunda derivada — consistência via lei de Ampère — revelaram o fator fixo. Ver a nota completa (com a cadeia de raciocínio) no topo de `scf/gradshafranov.py`.

Lição operacional: “resolve com a fórmula, confere com a mesma fórmula” não é validação independente, mesmo em arquivos/funções diferentes, se ambos herdaram a mesma normalização de uma derivação compartilhada.

Testes de normalização sem derivada — a defesa contra essa classe de erro:

$$\text{Fluxo: } u(\varpi, z) = \int_0^\varpi B_z(\varpi', z) \varpi' d\varpi' \quad (19)$$

$$\text{Ampère: } \oint_C \mathbf{B} \cdot d\mathbf{l} = 4\pi k_0 \int_S \rho \varpi dA \quad (20)$$

- **Ampère:** implementado e no repositório

→ `scf/tests/test_gradshafranov.py :: test_ampere_law()`

Usa um caso sintético com fonte de modo $l = 1$ puro (não a EOS/SCF completos), calcula $\oint \mathbf{B} \cdot d\mathbf{l}$ num laço retangular em (r, θ) e compara com $-\iint [\text{source}/\sin\theta] dr d\theta$ (forma equivalente derivada da lei de Ampère para esta geometria). Fecha a $\sim 2\%$.

- **Fluxo:** foi *proposto* durante a depuração (ver histórico da conversa que corrigiu o bug) mas **não foi implementado como teste separado** — o teste de Ampère sozinho já revelou e confirmou a correção do bug, então o teste de consistência de fluxo nunca chegou a ser escrito. Fica registrado como lacuna em §7.

1.5 Bernoulli e a parametrização

$$H + \Phi - M(u) = C, \quad M(u) = \int f(u) du \quad (21)$$

Com $f = k_0$ constante, $M(u) = k_0 u$.

A constante de integração é fixada **no centro**:

$$C = H(\rho_c) + \Phi_c - M(u_c) \quad (22)$$

onde Φ_c , u_c aqui denotam os valores no centro ($r = 0$) — **atenção:** este uso de u_c (valor de u no centro) é diferente do u_c usado em §1.9 (valor de u na última linha fechada, na superfície). São o mesmo símbolo para dois pontos de avaliação diferentes; o código nunca precisa dos dois ao mesmo tempo, mas o leitor deve notar a colisão. Como $\varpi = r \sin\theta$ se anula no centro para qualquer θ , $u_c(\text{centro}) = 0$ sempre e o termo $M(u_c)$ desaparece — ele fica na fórmula por generalidade, não porque contribua.

→ `scf/scf.py :: hachisu_scf()`

— linha $C = H_c + \Phi[0, 0] - M_u[0, 0]$.

Por que assim — a parametrização por (ρ_c, k_0)

A receita clássica de Hachisu fixa C impondo $H = 0$ em dois pontos da superfície (polo e equador). Isso é mal-posto para esta EOS: como $n \rightarrow 3$ no núcleo (§1.1), a massa fica quase independente da densidade central e o raio despenca, de modo que resolver para C a partir do raio inverte um mapa quase singular. Testado e confirmado neste projeto: a iteração de Picard sob essa receita é linearmente instável em toda a faixa $\rho_c = 10^6$ – 10^{10} g/cm³, com ou sem sub-relaxação.

A parametrização adotada é por (ρ_c, k_0) , duas entradas independentes, sem nenhuma condição de superfície — C é fixada por uma condição local (H no centro), não global (integral sobre toda a massa). Essa mudança por si só resolveu a instabilidade: convergência geométrica até precisão de máquina em ~ 12 iterações, contra divergência exponencial da receita antiga.

Ressalva: no regime de campo forte, quando o pico de densidade migra para fora do centro, esta âncora também perde o sentido físico que a torna estável — ver §6.

1.6 O loop SCF

1. $\rho \leftarrow$ palpite inicial (politropo esférico $n = 3$)
2. $A_\varphi \leftarrow 0$
3. **repetir:**
 4. $\Phi \leftarrow \text{Poisson}(\rho)$
 5. $A_\varphi \leftarrow \text{GradShafranov}(\rho, f)$
 6. $u \leftarrow \varpi A_\varphi$; $M \leftarrow \int f du$
 7. $C \leftarrow H(\rho_c) + \Phi_c - M(u_c)$
 8. $H \leftarrow C - \Phi + M(u)$
 9. $\rho_{\text{novo}} \leftarrow \text{EOS}^{-1}(H)$ $[H \leq 0 \Rightarrow \rho = 0]$
 10. $\rho \leftarrow (1 - \omega)\rho + \omega\rho_{\text{novo}}$ $\omega \approx 0,3$
11. **até** $\max |\Delta\rho|/\rho_c < \text{tol}$

→ `scf/scf.py :: hachisu_scf()`

implementa os passos 1, 3–9 e 11 literalmente (passo 2 é implícito: u começa em zero antes da primeira iteração).

Discrepância com o código real, passo 10: a implementação **não faz sub-relaxação**. A linha correspondente em `hachisu_scf()` é `rho = rho_new` — substituição direta, equivalente a $\omega = 1$, não $\omega \approx 0,3$. Não há parâmetro ω na assinatura da função. Isso não é um descuido: é consequência direta da mudança descrita no quadro de §1.5. A receita antiga (duas condições de superfície) exigia sub-relaxação para tentar estabilizar uma iteração que era instável de qualquer forma; a parametrização (ρ_c, k_0) converge por substituição direta porque a instabilidade que a sub-relaxação tentava mascarar foi removida na raiz. Ver a nota de projeto no topo de `scf/scf.py` para o histórico completo (inclusive testes que mostraram que sub-relaxar a receita antiga não resolvia o problema).

O critério de convergência (`tol`) é um parâmetro numérico, não físico — o dashboard expõe valores de 10^{-4} a 10^{-8} (Aba 1), com 10^{-6} como padrão.

1.7 Virial e diagnósticos de energia

$$W = \frac{1}{2} \int \rho \Phi dV \quad (\text{gravitacional, negativa}) \quad (23)$$

$$\Pi = \int P dV \quad (\text{interna}) \quad (24)$$

$$E_{pol} = \int \frac{B_r^2 + B_\theta^2}{8\pi} dV \quad (25)$$

$$E_{tor} = \int \frac{B_\varphi^2}{8\pi} dV \quad (26)$$

$$\mathcal{M} = E_{pol} + E_{tor} \quad (27)$$

→ `scf/diagnostics.py :: gravitational_energy()`

(W), `pressure_integral()` (Π), `magnetic_energies()` (E_{pol} , E_{tor} , $\mathcal{M}$ — chamada de `E_mag` no código). Todas usam `volume_integral()` como base ($dV = r^2 \sin \theta dr d\theta d\varphi$, φ integrado analiticamente para 2π).

Teorema do virial escalar para configuração estática:

$$W + 3\Pi + \mathcal{M} = 0 \quad (28)$$

Erro virial, usado como portão de qualidade:

$$VE = \frac{|W + 3\Pi + \mathcal{M}|}{|W|} \quad \text{critério de aceite: } VE < 10^{-3} \quad (29)$$

→ `scf/diagnostics.py :: virial_error()`

O limite 10^{-3} é convenção de projeto (V3 do plano), não uma constante física — aparece hard-coded como comparação em `dashboard/pages/1_equilibrio.py` e `3_exportacao.py`, não em `units.py` nem em `diagnostics.py` (nenhum módulo de física define esse limite como constante nomeada — é uma decisão de UI/gate repetida em duas páginas).

Identidade do virial magnético. Exata, e é o teste que liga o setor magnético ao gravitacional:

$$\int \rho \nabla M(u) \cdot \mathbf{r} dV = \int \frac{B^2}{8\pi} dV \quad (30)$$

Esta identidade **não corresponde a nenhuma função no código**. Foi verificada numericamente de forma ad hoc durante a depuração do bug de §1.4 (comparando $k_0 \int \rho r \partial u / \partial r dV$ com $\int B^2 / 8\pi dV$ para uma solução convergida), mas não existe uma função em `scf/diagnostics.py` que a calcule nem um teste comitado que a exercite. Fica em §7 como lacuna.

Nota conceitual obrigatória

$M(u)$ **não é** a energia magnética. É o potencial específico da força de Lorentz — apenas a parcela que entra no Bernoulli. Não existe identidade *local* entre $M(u)$ e $B^2/8\pi$. A identidade *global* acima é o que força as duas quantidades a concordarem em ordem de grandeza. Medida neste projeto, no regime linear (campo perturbativo), a razão $(E_{mag}/|W|)/(M_u/H_c)$ vale $\approx 0,5$ e é constante em k_0 (confirmado dobrando k_0 : ambas as razões quadruplicam, a razão entre elas não muda); ela desliza para $\sim 0,38$ conforme o campo deixa de ser perturbação pequena ($k_0 \gtrsim 10^{-12}$ na configuração $\rho_c = 10^9$, $R \approx 3 \times 10^8$).

cm). Essa foi a observação que, junto com o teste de Ampère, confirmou que o bug do quadro de §1.4 era real e não um artefato de unidades.

Limite físico. $\mathcal{M} < |W|$ é vínculo rígido: $E_{mag}/|W| \geq 1$ é configuração impossível (violaria o virial com $\Pi \geq 0$). Use como âncora de sanidade ao ler qualquer resultado — se o dashboard mostrar $E_{mag}/|W|$ maior que alguns décimos, desconfie antes de acreditar.

1.8 As duas razões Bt/Bp

$$\left(\frac{B_t}{B_p}\right)_{\text{energia}} = \frac{E_{tor}}{E_{pol}} \quad (31)$$

$$\left(\frac{B_t}{B_p}\right)_{\text{amplitude}} = \frac{\max |B_\varphi|}{\max |B_{pol}|} \quad (32)$$

→ `scf/toroidal.py :: bt_bp_ratios()`

Diferem por ordens de grandeza porque o toroidal está confinado a um volume pequeno (§1.9). A literatura frequentemente não diz qual usa. **O dashboard sempre mostra as duas, rotuladas** (Aba 1 e Aba 3).

1.9 O campo toroidal e o twisted torus

Na formulação barotrópica, $\beta = \beta(u)$: a função toroidal depende só da função de fluxo. É essa condição que anula a componente φ da força de Lorentz e permite a existência da integral de Bernoulli (§1.5).

Consequência geométrica: fora da estrela não há corrente, logo $B_\varphi = 0$ lá; como $\beta = \beta(u)$, ela precisa se anular em toda linha de fluxo que escapa da superfície. **O toroidal fica automaticamente confinado à região de linhas poloidais fechadas** — o toro torcido não é imposto por decreto geométrico, ele cai da consistência da equação.

Forma funcional adotada:

$$\beta(u) = \zeta (u - u_c)^{m+1} \Theta(u - u_c) , \quad m \geq 1 \quad (33)$$

com u_c o valor de u na última linha fechada (aqui, u_c = valor de u na superfície — ver a distinção de notação em §1.5). O expoente ≥ 1 mantém $\beta\beta'$ contínua na borda do toro.

→ `scf/toroidal.py :: impose_toroidal()`

implementa esta forma já substituída em $B_\varphi = \beta/\varpi$: $B_\varphi = \zeta(u - u_c)^{m+1}/\varpi$ para $u > u_c$, 0 fora. `find_uc()` implementa a busca de u_c como o **máximo de u ao longo de toda a superfície estelar** ($H = 0$, varrendo θ do polo ao equador) — é essa escolha específica de “última linha fechada” que o código usa; contornos com u maior que esse máximo, por definição, não tocam a superfície em nenhum ponto e ficam inteiramente internos.

ζ não é um parâmetro de entrada direto do usuário — o dashboard pede a razão B_t/B_p (energia) alvo e resolve ζ para atingi-la, aproveitando que B_φ é linear em ζ (logo E_{tor} é quadrático):

→ `scf/toroidal.py :: solve_zeta_for_energy_ratio()`

Por que o toroidal é imposto e não resolvido

A condição $\beta = \beta(u)$ confina o toroidal a um toro de volume pequeno, e a razão de energias resultante — se β fosse extraída de uma condição de fechamento barotrópica geral em vez de escolhida livremente — sai em poucos por cento. **Não é possível atingir $B_t/B_p \sim 1/2$ por essa via.** Por isso o projeto resolve o SCF barotrópico apenas para o poloidal (§1.4–§1.6, com $f(u) = k_0$) e impõe o toroidal por cima, com a razão desejada (ζ resolvida para o alvo), carregando no Castro fora de equilíbrio exato e relaxando com amortecimento (sponge, Aba 3). Para estudo dinâmico não é preciso equilíbrio exato — basta estar perto o suficiente para o transiente não destruir a topologia.

Fração de volume do toro (quanto da estrela tem $u > u_c$):

→ `scf/toroidal.py :: closed_torus_volume_fraction()`

Espessura radial do toro (usada para checar resolução de malha, Aba 3):

→ `scf/toroidal.py :: torus_radial_extent()`

, medida ao longo do equador por padrão.

1.10 Escalas de tempo e unidades

$$t_{din} = \sqrt{\frac{R_{eq}^3}{GM}} \quad (34)$$

$$v_A = \frac{\langle B \rangle}{\sqrt{4\pi\bar{\rho}}} \quad (35)$$

$$t_{\text{Alfvén}} = \frac{R_{eq}}{v_A} \quad (36)$$

→ `dashboard/units.py :: dynamical_time(), alfven_speed(), alfven_time()`

$\langle B \rangle$ e $\bar{\rho}$ (médias volumétricas) são calculadas em `dashboard/pages/3_exportacao.py` diretamente com `diagnostics.volume_integral()` — não há uma função `mean_field()` dedicada em `scf/`.

A razão $t_{\text{Alfvén}}/t_{din}$ mede o custo da simulação: no regime de campo fraco ela chega a 10^3 – 10^4 , que é proibitivo; no regime de campo forte deste projeto, com $E_{mag}/|W| \sim 0,1$, ela fica em 2–3 (D4 do plano).

Unidade natural de campo. De $B^2/8\pi \sim G\rho^2 R^2$:

$$B_{unit} = R\rho\sqrt{8\pi G} \quad (37)$$

Para $\rho_c \sim 10^9$ e $R \sim 10^8$ cm isso dá $\sim 10^{14}$ G, que é também a ordem do campo virial de uma anã branca. **Esta fórmula não está implementada em nenhum lugar do código** — foi usada apenas como estimativa de ordem de grandeza durante a depuração do bug de §1.4, para checar se o campo relatado fazia sentido físico antes de procurar o erro numérico. **Âncora de sanidade:** com $E_{mag}/|W| \sim 0,1$, o campo exibido pelo dashboard deve estar na casa de 10^{13} – 10^{14} G; se aparecer muito diferente disso, desconfie antes de acreditar (foi exatamente essa desconfiança que achou o bug).

Convenção do Castro: o campo é carregado como $B' = B/\sqrt{4\pi}$. O dashboard exibe **sempre em gauss** e converte apenas na exportação (Aba 3).

→ `dashboard/units.py :: gauss_to_castro(), castro_to_gauss()`

Conferido neste ciclo de trabalho: o fator bate com o plano ($\sqrt{4\pi} \approx 3,5449$). **Nota de estado:** `gauss_to_castro()` existe e está correta, mas **não é chamada em nenhum lugar do pipeline de exportação atual** (`dashboard/pages/3_exportacao.py` escreve `B_phi` no HDF5 diretamente em gauss, com um atributo `units` explícito documentando isso). A conversão para a convenção B' do Castro é responsabilidade do `problem_initialize.H` do lado do Castro (ainda não escrito — Fase 0 do plano pendente), que lerá o HDF5 em gauss e aplicará `gauss_to_castro()` (ou o equivalente em C++) ao montar o estado interno. Ver §7.

Todas as conversões de unidade de exibição (gauss, km) — tanto o número quanto a formatação da string — vivem em `dashboard/units.py`, ponto único de verdade (regra R4 do dashboard): `cm_to_km()`, `g_to_msun()`, `format_gauss()`, `format_km()`, `format_km_value()`.

2 Aba 1 — Equilíbrio

→ `dashboard/pages/1_equilibrio.py`

Executa uma corrida única do SCF (§1.6) e mostra o resultado. Usa todo o núcleo teórico (§1.1–1.10).

Parâmetros de entrada (barra lateral): ρ_c , μ_e , k_0 (opcional, liga/desliga campo poloidal), razão B_t/B_p alvo e m (toroidal, opcional), mais os parâmetros numéricos da malha (N_r , N_θ , l_{max} , tol , max_iter). θ sempre cobre $[0, \pi]$ inteiro — sem simetria equatorial, decisão do plano (D3) para não mascarar modos assimétricos ($m = 1$).

Faixa de k_0 . Não é conhecida a priori (depende de ρ_c , R , μ_e de forma não trivial — ver o bug de §1.4, que por muito tempo fez a faixa aparente parecer errada por 3–4 ordens de grandeza). O dashboard sonda empiricamente subindo k_0 geometricamente até $VE > 10^{-3}$ ou a SCF parar de convergir, numa malha grosseira, e grava o resultado num cache em `dashboard/k0_range_cache.json`, indexado por (ρ_c, μ_e) .

→ `dashboard/pages/1_equilibrio.py :: _estimate_k0_max()`

Escalares exibidos (tabela “Escalares”):

Escalar na tela	Definição	Função
$M/M_\odot$	massa total / <code>M_SUN</code>	<code>scf.total_mass()</code> , <code>units.M_SUN</code>
R_{eq} , R_{pol} (km)	raio onde $\rho \rightarrow 0$, no equador e no polo	<code>diagnostics.equatorial_polar_radii()</code>
R_{pol}/R_{eq}	achatamento	razão direta
ρ_c confirmado	$\rho[r=0]$ pós-convergência	deve bater com o ρ_c de entrada
$\bar{\rho}$ média	M/volume do elipsoide	aproximação geométrica, não integral exata
W , $E_{int}=\Pi$, E_{mag} , E_{pol} , E_{tor} $E_{mag}/ W $	$(R_{eq}, R_{eq}, R_{pol})$ §1.7	<code>diagnostics.virial_error()</code> , <code>magnetic_energies()</code>
$B_{pol,max}$, $B_{central}$, $B_{tor,max}$	intensidade de campo adimensional	razão direta (âncora de sanidade, §1.7/1.10)
fração de volume do toro	valores de campo, gauss	máximos/pontuais de <code>Br</code> , <code>Bth</code> , <code>Bphi</code>
VE	§1.9	<code>toroidal.closed_torus_volume_fraction()</code>
	§1.7	<code>diagnostics.virial_error()</code>

Convergência: dois gráficos ($\max |\Delta\rho|/\rho_c$ e VE , ambos por iteração, escala log)

→ `plots.plot_convergence()`, `plots.plot_virial_history()`

O histórico de VE só existe se `hachisu_scf(..., track_virial=True)` — custa integrais extras a cada iteração, por isso é opcional (ver §1.6).

Bt/Bp: as duas razões de §1.8, lado a lado, sempre rotuladas.

Figuras do plano meridional (§1.3):

- **densidade:** mapa de cor de ρ , com a fronteira $H = 0$ (a superfície da estrela) sobreposta em ciano

→ `plots.plot_density()`

- **linhas de campo poloidal:** contornos de u — fisicamente, cada contorno é uma linha de campo poloidal (§1.3), porque $\mathbf{B}$ é tangente às curvas de u constante por construção ($\mathbf{B} \cdot \nabla u = 0$ decorre diretamente das fórmulas de B_r, B_θ em termos de derivadas de u). A

última linha fechada (u_c , §1.9) é destacada em vermelho quando há toroidal imposto

→ `plots.plot_flux_contours()`

- **campo toroidal:** mapa de cor de B_φ , mostra o toro confinado

→ `plots.plot_toroidal()`

Todos os eixos radiais em km, campo sempre em gauss com colorbar em notação científica (regra R4, §1.10).

3 Aba 2 — Varredura

→ `dashboard/pages/2_varredura.py`, `dashboard/sweep_worker.py`

Roda o SCF (§1.6) numa grade de (ρ_c, k_0) em paralelo (`ProcessPoolExecutor`), com cache por hash dos parâmetros (`store.run_exists()/store.save_run()`, §5). Cada ponto da grade é uma chamada independente a `scf.hachisu_scf()` seguida dos mesmos diagnósticos da Aba 1 — nenhuma física nova, só orquestração.

→ `dashboard/sweep_worker.py :: run_one()`

— a função picklable que cada processo da grade executa.

O que é uma sequência de equilíbrio. Fixando ρ_c e variando k_0 (ou vice-versa), obtém-se uma sequência de configurações de equilíbrio. **A sequência termina** quando o SCF para de convergir ou quando VE ultrapassa o limite de aceite (§1.7) e não melhora com resolução — ver os números medidos em §6. Uma terminação de sequência é, em si, um resultado físico (não um erro a ser corrigido): sinaliza o limite de validade daquela família de equilíbrios.

Diagrama M-R. R_{eq} (km) no eixo x, $M/M_\odot$ no eixo y, colorido por k_0 . A reta horizontal em $1,44 M_\odot$ marca o limite de Chandrasekhar **sem campo** ($\mu_e = 2$) — não é o limite físico da sequência magnetizada, é só uma referência de leitura. Overlay opcional de pontos da literatura de `dashboard/data/referencias/bera_bhattacharya_2014.csv`, se o arquivo existir (não existe atualmente neste repositório — nenhum dado foi digitalizado; o dashboard funciona sem ele e avisa que falta).

Por que o máximo de massa relevante é sobre o plano inteiro, não uma fatia

Este é o ponto que mais gera comparação errada com a literatura. $M_{max}(k_0=0)$ já depende de ρ_c : só se aproxima do limite de Chandrasekhar assintoticamente, para ρ_c alto ($\sim 10^{11} - 10^{12} \text{ g/cm}^3$ — ver `scf/tests/test_scf_v1.py`, validado a 0,78% nessa faixa). Uma fatia de ρ_c baixo (por exemplo $\rho_c = 10^9$, usada durante a depuração deste projeto) dá $M(k_0=0) = 1,39 M_\odot$ — **já abaixo** do limite de Chandrasekhar sem campo, e qualquer massa medida ao longo dessa fatia (com ou sem campo) não é comparável ao número de referência da literatura ($M_{max} \sim 1,9 M_\odot$ em Bera & Bhattacharya 2014), que é o máximo tomado sobre **todo** o plano (ρ_c, k_0) , não sobre uma reta k_0 variável com ρ_c fixo. A varredura desta aba é exatamente o que permite fazer essa comparação corretamente — mapeando o plano, não uma fatia.

Convergência da grade. Pontos que não convergem são registrados (não descartados silenciosamente) e mostrados num expensor separado; a taxa de convergência da grade é, ela mesma, informação sobre onde a família de soluções termina.

Mapa de calor de VE. Sobre a grade (ρ_c, k_0) , em $\log_{10}(\text{VE})$ — revela onde o método está no limite da validade sem precisar ler número por número.

4 Aba 3 — Exportação

→ `dashboard/pages/3_exportacao.py`

Pega um equilíbrio já salvo (Aba 1 ou 2), impõe o toroidal (§1.9) e gera os três artefatos de saída: HDF5 de dado inicial, inputs do Castro, `run_manifest.json`.

Imposição do toroidal. Mesma função de §1.9 (`toroidal.solve_zeta_for_energy_ratio()`), com controles próprios desta aba (a razão alvo pode ser diferente da usada quando o equilíbrio foi salvo). A continuidade de $\beta\beta'$ na borda do toro é **garantida analiticamente** para $m \geq 1$ (não é recalculada numericamente aqui — é uma propriedade da forma funcional de §1.9, $(u - u_c)^{m+1}$ e sua derivada vão a zero em $u = u_c$ para qualquer $m \geq 1$); a página só confirma que $m_{\text{tor}} \geq 1$ foi respeitado.

Por que o campo é inicializado pelo potencial vetor, não por $\mathbf{B}$ nos centros. Isto é uma decisão do lado Castro (D3 do plano, seção 5), não algo que o SCF resolve — mas o dashboard já prepara os dados nesse formato: o HDF5 exportado contém A_φ (calculado como u/ϖ a partir da função de fluxo convergida), **não** B_r , B_θ diretamente. O algoritmo de transporte restrito do Castro mantém $\mathbf{B}$ nas faces da malha; inicializar via $\nabla \times \mathbf{A}$ interpolado nas arestas garante $\nabla \cdot \mathbf{B} = 0$ na precisão de máquina por construção. Inicializar com valores de $\mathbf{B}$ nos centros das células não dá essa garantia. **O que falta:** a interpolação de A_φ para as arestas da malha cartesiana e o cálculo de $\nabla \times \mathbf{A}$ ali é responsabilidade do `problem_initialize_mhd_data.H` do Castro, que ainda não foi escrito (Fase 0 do plano pendente) — o dashboard entrega A_φ numa malha esférica (r, θ) ; a interpolação para a malha cartesiana do Castro acontece do outro lado.

Convenção $B' = B/\sqrt{4\pi}$. Ver §1.10. O HDF5 exportado por esta aba guarda B_φ em gauss puro, com um atributo `units` no cabeçalho dizendo isso explicitamente — a conversão para a convenção do Castro ainda não acontece neste pipeline (ver nota de estado em §1.10 e a lacuna em §7).

Escalas derivadas e o custo da simulação. t_{din} , v_A , $t_{\text{Alfvén}}$ e a razão $t_{\text{Alfvén}}/t_{\text{din}}$ de §1.10, calculados a partir do $\langle B \rangle$ e $\bar{\rho}$ do equilíbrio carregado. Um badge de sucesso aparece quando a razão está entre 0,3 e 3 — o regime de campo forte que torna a simulação barata (D4 do plano).

Checagem de resolução do toro. Dado o número de células por lado da caixa do Castro (128/256/384) e o tamanho da caixa (múltiplo de R_{eq}), calcula quantas células atravessam a espessura radial do toro (`toroidal.torus_radial_extent()`, medida no equador). Menos de 10 células gera aviso — o toro é pequeno comparado à estrela e é ele que precisa ser resolvido (plano, seção 6).

Parâmetros da caixa. `castro.small_dens` é escolhido como $\rho_c \times 10^{-8}$ — esta é uma **convenção de projeto deste dashboard**, não um valor derivado da física nem citado no plano; não há justificativa teórica documentada para o expoente 10^{-8} além de ser um piso de densidade suficientemente baixo para não perturbar a estrela e suficientemente alto para não gerar v_A absurdo no vácuo numérico (o “problema do fluff”, plano seção 5). O raio de início do sponge ($1,5 R_{eq}$), a duração do amortecimento ($5 t_{\text{din}}$) e a amplitude da perturbação ($10^{-4} c_s$, usando `eos.sound_speed()` no centro) vêm diretamente dos valores citados no plano (seção 5), não são recalculados/otimizados por este dashboard.

R5 — exportação bloqueada se $VE \geq 10^{-3}$, sem opção de forçar. Implementado como um `if/else` simples em torno do botão de exportação — não há mecanismo de override em nenhuma camada.

5 Aba 4 — Registro

→ `dashboard/pages/4_corridas.py`, `dashboard/store.py`

Sem física — este módulo só persiste o que as outras abas já calcularam (regra R1/R3 do dashboard).

O que é gravado, por corrida, em `dashboard/runs/<hash>/`:

Arquivo	Conteúdo	Função
<code>params.json</code>	parâmetros de entrada completos	<code>store.save_run()</code>
<code>scalars.json</code>	escalares derivados (mesmos da tabela da Aba 1)	idem
<code>fields.npz</code>	ρ , Φ , u , H , B_{Φ} , r , θ na malha	idem
<code>manifest.json</code>	hash, timestamp, git, dependências	idem

`dashboard/runs/index.csv` agrega hash + parâmetros + escalares de todas as corridas, para a Aba 4 carregar rápido sem abrir cada diretório (`store.load_index()`).

Por que hash de git importa. `manifest.json` grava `git_commit_hash(REPO_ROOT)` — o commit de `scf/` e `dashboard/` (o mesmo repositório git cobre os dois, inicializado especificamente para dar proveniência a este dashboard; `amrex/`, `castro/`, `microphysics/` são excluídos via `.gitignore`, são repositórios próprios). Um escalar sem o commit associado não pode ser reproduzido com confiança — se o código mudar (por exemplo, o bug de §1.4 sendo corrigido), resultados salvos antes e depois **não são comparáveis**, mesmo com os mesmos parâmetros de entrada. `git_dirty()` também é gravado — sinaliza se havia mudanças não commitadas no momento da corrida, o que torna a reprodução exata impossível mesmo sabendo o hash.

→ `dashboard/store.py :: git_commit_hash(), git_dirty(), dependency_versions()`

(versões de `numpy`, `scipy`, `streamlit`, `plotly`, `h5py`, `python`).

Cache. O hash é `sha256(json(params, sort_keys=True))[:12]` (`store.params_hash()`) — determinístico nos parâmetros, usado tanto para nomear o diretório da corrida quanto para a Aba 2 pular pontos já calculados.

Versão de esquema: **não existe** um campo `schema_version` (ou equivalente) em `manifest.json` ou `scalars.json` atualmente. Quando o conjunto de escalares mudou durante este projeto (por exemplo, ao adicionar `B_pol,max` (G) ao schema da Aba 2), corridas salvas antes da mudança ficaram com colunas ausentes em `index.csv`, quebrando gráficos que esperavam a coluna nova — isso já aconteceu de verdade durante o desenvolvimento e foi contornado apagando o cache antigo manualmente, não por um mecanismo de versionamento. Fica registrado como lacuna em §7.

Funcionalidades: tabela filtrável/ordenável (`st.data_editor`, formatada por coluna conforme a regra R4 — gauss em notação científica, km com 2 casas), comparação lado a lado de duas corridas, recarregar uma corrida na Aba 1 (via `st.session_state["reload_run_params"]` + `st.switch_page`), marcar como referência (`store.mark_reference()` — hoje só grava a flag; nenhuma outra aba lê `reference` ainda, ver §7).

6 Limitações conhecidas

Números medidos, não descrições vagas — todos na configuração $\rho_c = 10^9$ g/cm³, $R \approx 3 \times 10^8$ cm, $nr=161$, $ntheta=65$, $l_{max} = 16$ salvo onde indicado:

- **O critério $VE < 10^{-3}$ falha acima de $k_0 \approx 2 \times 10^{-12}$ nesta configuração.** O último ponto válido é $k_0 \approx 1,6 \times 10^{-12}$, $M \approx 1,50 M_\odot$, $VE \approx 6,6 \times 10^{-4}$. Em $k_0 \approx 2,3 \times 10^{-12}$ ($M \approx 2,02 M_\odot$), $VE \approx 1,57 \times 10^{-3}$ — acima do limite.
- **Nesse mesmo ponto ($k_0 \approx 2,3 \times 10^{-12}$) o pico de densidade migra para fora do centro** — de $r_{\text{idx}}=1$ (essencialmente a origem) para $r_{\text{idx}} \approx 25$ (raio real, não artefato de grade) — e $R_{\text{pot}}/R_{\text{eq}}$ cai a $\sim 0,61$: evacuação equatorial genuína. A âncora em $\rho_c(r=0)$ (§1.5) perde sentido físico exatamente aí, porque o centro deixou de ser o ponto de densidade máxima.
- **Está em aberto se a falha do VE acima é resolução insuficiente ou terminação de sequência genuína — mas há evidência forte para a segunda hipótese:** um estudo de convergência feito neste projeto (l_{max} de 16 a 48, malha de 129^2 a 385^2 , quase $9\times$ mais pontos) manteve VE em $1,2\text{--}1,6 \times 10^{-3}$, **sem tendência de queda** — um platô acima do limite, não uma curva convergindo para zero. Isso é a assinatura esperada de uma terminação física, não de sub-resolução.
- **Todos os resultados de sequência (k_0 variável) documentados acima são de uma única fatia $\rho_c = 10^9$,** onde $M(k_0=0) = 1,39 M_\odot$ — abaixo do próprio limite de Chandrasekhar sem campo ($1,44 M_\odot$). Comparações com máximos de massa da literatura ($M_{\text{max}} \sim 1,9 M_\odot$, Bera & Bhattacharya 2014) exigem varredura no plano (ρ_c, k_0) inteiro (Aba 2), não uma fatia — ver §3.
- **A conversão $B' = B/\sqrt{4\pi}$ do Castro não é aplicada em nenhum lugar do pipeline de exportação atual (§1.10, §4)** — o HDF5 exportado guarda B em gauss puro, documentado via atributo. Fica responsabilidade do `problem_initialize.H` do Castro (não escrito ainda).
- **A Fase 0 do plano (compilar o Castro com `USE_MHD=TRUE`) está pendente** — dependências de sistema (`gfortran`, `libhdf5-openmpi-dev`, `libopenmpi-dev`) ainda não instaladas neste ambiente. Nada na Aba 3 foi testado contra um Castro real.

7 Questões em aberto

Lacunas identificadas ao escrever este documento — não preenchidas por conta própria (regra G1):

1. **Teste de consistência de fluxo** ($u(\varpi, z) = \int_0^\varpi B_z \varpi' d\varpi'$, §1.4) foi proposto durante a depuração do bug da função de Green mas nunca implementado como teste separado. O teste de Ampère sozinho bastou para achar e confirmar o bug; o de fluxo ficaria como segunda linha de defesa independente.
2. **Identidade do virial magnético** ($\int \rho \nabla M(u) \cdot \mathbf{r} dV = \int B^2/8\pi dV$, §1.7) não tem função dedicada em `diagnostics.py` nem teste comitado — só foi verificada numericamente de forma ad hoc numa sessão de depuração.
3. **A fórmula $J_\varphi = c\rho\varpi f(u)$ (§1.4) não corresponde a nenhuma função** — nunca é calculada como quantidade nomeada no código.
4. $B_{\text{unit}} = R\rho\sqrt{8\pi G}$ (§1.10, unidade natural de campo) não está implementada — foi usada só como estimativa de ordem de grandeza durante a depuração.
5. **Sem `schema_version` em `manifest.json/scalars.json` (§5)** — mudanças no conjunto de escalares já quebraram `index.csv` de corridas antigas uma vez durante este projeto.

6. `store.mark_reference()` grava a flag `reference` no índice, mas nenhuma outra aba (em particular a Aba 2, que segundo o prompt original deveria mostrar corridas de referência nos gráficos) lê essa flag ainda.
7. `castro.small_dens` = $\rho_c \times 10^{-8}$ (§4) é uma convenção sem justificativa teórica documentada — funciona como ponto de partida, não como valor calibrado.
8. **A questão original que motivou toda a investigação do bug de §1.4** — se a razão $(E_{mag}/|W|)/(M_u/H_c) \approx 0,5$ (regime linear) tem uma derivação fechada, ou é só o que se observa numericamente nesta EOS e nesta faixa de parâmetros — não foi resolvida analiticamente, só confirmada como autoconsistente (constante sob $k_0 \rightarrow 2k_0$).
9. `gauss_to_castro()/castro_to_gauss()` existem e estão corretas (conferido neste ciclo), mas não são chamadas por nenhum código de exportação ainda — ver limitação em §6.

8 Referências

Método SCF e equilíbrios magnetizados

- Hachisu, I. 1986, ApJS 61, 479 — o método SCF
- Tomimura, Y. & Eriguchi, Y. 2005, MNRAS — twisted torus, referência canônica
- Lander, S. K. & Jones, D. I. 2009, MNRAS — campos mistos, funções livres
- Lander, S. K. & Jones, D. I. 2012 — estabilidade de campos mistos

Anãs brancas magnetizadas

- Das, U. & Mukhopadhyay, B. 2014, MNRAS 445, 3951 — SCF para AB magnetizada
- Bera, P. & Bhattacharya, D. 2014, MNRAS — M-R autoconsistente com Lorentz
- Bera, P. & Bhattacharya, D. 2016, MNRAS 456, 3375 — geometria de campo
- Bera, P. & Bhattacharya, D. 2017, MNRAS 465, 4026 — estudo de perturbação
- Nityananda, R. & Konar, S. 2014 — crítica aos modelos super-Chandrasekhar
- Coelho, J. G. et al. 2014 — limites do virial

Estabilidade

- Markey, P. & Tayler, R. J. 1973 — a instabilidade $m = 1$
- Braithwaite, J. & Spruit, H. C. 2004 — relaxação para configuração estável
- Braithwaite, J. & Nordlund, Å. 2006

Códigos

- Castro: <https://github.com/AMReX-Astro/Castro>
- Documentação MHD: <https://amrex-astro.github.io/Castro/docs/mhd.html>
- XNS: <https://www.arcetri.inaf.it/science/ahead/XNS/>